



AT ALL NETWORKS

Object-Oriented Programming

Program



AT ALL NETWORKS

Classes, objects & Encapsulation

Inheritance & Polymorphism

Composition, Interfaces & special methods

SOLID principle & clean code

Design Patterns

Testing & Project structure





AT ALL NETWORKS

OOP fundamentals

Classes & Objects



AT ALL NETWORKS

Object-Oriented Programming (OOP) allows the program to be structured in the form of **objects** like those found in everyday life. The **class** defines the **structure** of **objects** and contains attributes and methods.

The **class** defines the **structure**, it describes what **attributes** and **methods** its **objects** will have.

- **attributes** are the data that the object will hold.
- **methods** are the behaviors of the object.



The **class** is the **blueprint**.

An **instance** is the **real object** built from it.

Procedural vs OOP



AT ALL NETWORKS

Procedural

Functions

Top-down - Break a problem into steps

Separated - data passed between functions

None - everything is accessible

Limited - duplicate or adapt functions

Scripts, small tools, linear workflow

C Pascal FORTRAN

UNIT OF CODE

APPROACH

DATA & LOGIC

ACCESS CONTROL

REUSABILITY

BEST SUITED FOR

EXAMPLES

Object-Oriented

Objects (data + behavior together)

Bottom-up – model the problem as entities

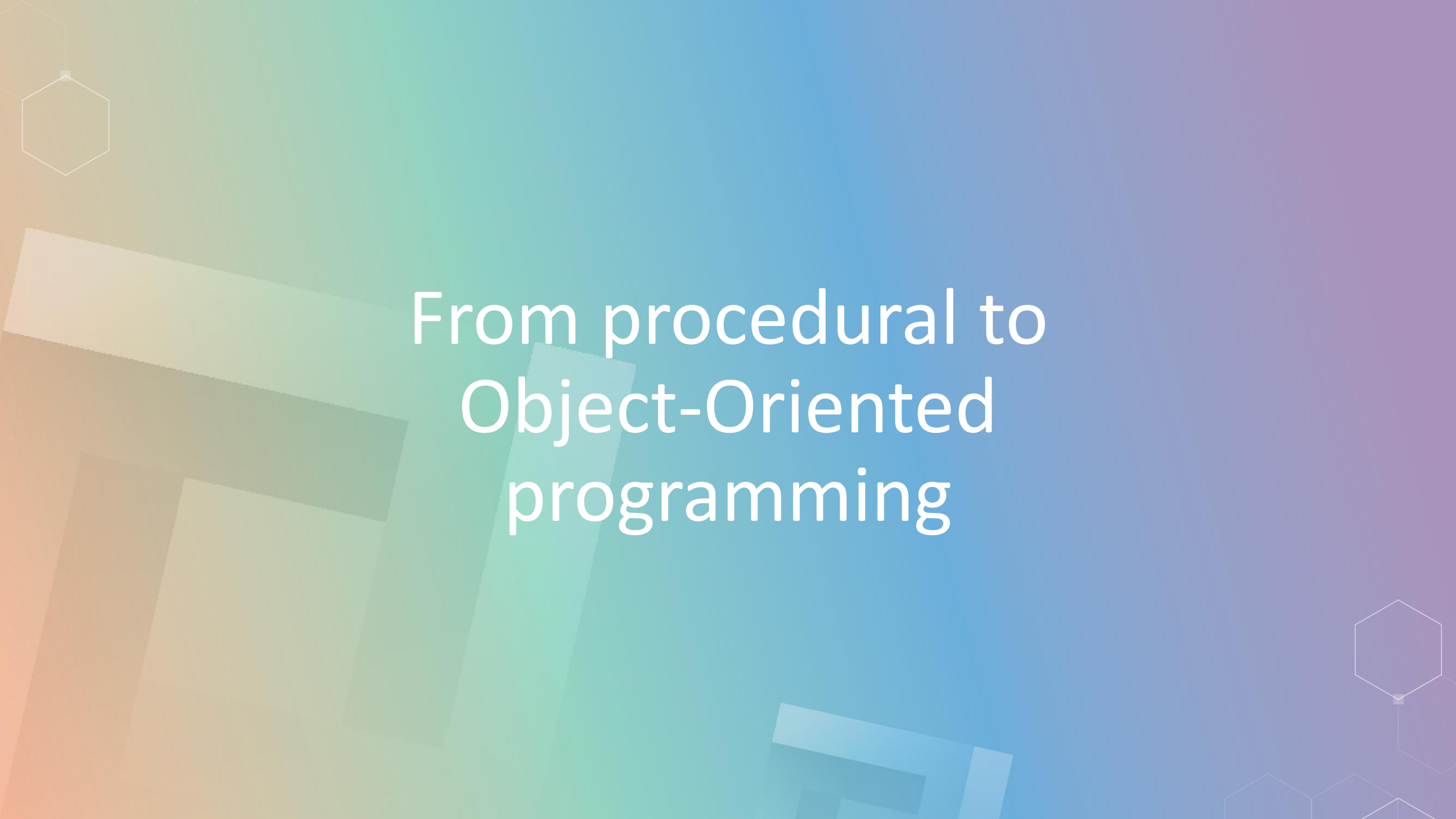
Bundled – an object owns its data and methods

Encapsulation – control what's exposed

High – inheritance and composition

Larger systems, maintainable codebases

Python Java C++



From procedural to Object-Oriented programming

Classes



AT ALL NETWORKS

Definition of a class

```
class Animal:  
  
    def __init__(self):  
        pass
```

class is the keyword to define a class.

The **`__init__`** method is the **initializer** method. It is called automatically when a new **instance** is created, and its job is to set the initial state of the object.

The **constructor** in Python is **`__new__`** but its almost never used except for example in a **Singleton Pattern**

In Python, **`self`** is the **first parameter of every instance method**. It refers to the specific object the method is called on. Python passes it automatically – you declare it, but you never pass it yourself.

Objects



AT ALL NETWORKS



Every **instance** is an **object**, but not every **object** is an **instance** (of a user-defined class).

Instantiation of an object

Instantiate an object in Python is done by calling the class like a function, which trigger the `__init__` to initialize the object.

```
animal1 = Animal()
```

Order of

Class definition

Defining class with attributes & methods

Calling the class

`obj = ClassName(args)`
creates a new object

`__new__` method

allocates memory

`__init__` method

initialize object's attributes

Object reference

The variable refers to the created instance

Getting the type of an object

```
ani1 = Animal()
ani2 = Animal()
print(ani1 is ani2)           # False
print(type(ani1))             # <class '__main__.Animal'>
print(isinstance(ani1, Animal)) # True
```



An instance of a **subclass** returns **True** for the parent class too !

Attributes



AT ALL NETWORKS

```
class Animal:

    latin_name = "Animalia"
    science = "Zoology"
    is_vegetal = False

    def __init__(self, name, legs_number, marin):
        self.name = name
        self.legs_number = legs_number
        self.marin = marin

animal1 = Animal('Dog', 4, False)

print(animal1)
print(animal1.name)
```

```
<__main__.Animal object at 0x000002C3E9397E00>
Dog
```

A **class attribute** is common to all objects.

If its value is modified, the change will apply to **all instances** of that class.

A class attribute is defined **directly in the class body**, outside any method.

An **instance attribute** is specific to each object.

If its value is modified, the change will only apply to **the instance** in question.

An instance attribute is defined **inside __init__**, via **self**.

Attributes



AT ALL NETWORKS

```
animal1 = Animal('Dog', 4, False)

print(animal1)
print(animal1.name)
```

The `.` is the way to reach **inside** the object.

```
<__main__.Animal object at 0x000002C3E9397E00>
Dog
```

What you're accessing	Syntax	Example
Instance attribute	object.attribute	animal1.name
Instance method	object.method()	animal1.feed("banana")
Class attribute	Class.attribute	Animal.latin_name
Class method	Class.method()	Animal.get_animal_count()

Common bug patterns



AT ALL NETWORKS

The shadowing trap

To **modify a class attribute**, always **go through the class**, never through an instance. If you assign via an instance, Python doesn't modify the shared value – it **creates a new instance attribute that hides the class attribute**. From that point on, that instance will always read its own value and ignore the class attribute.

```
class Product:  
    discount = 0.10
```

```
p1 = Product()  
p2 = Product()
```

```
p1.discount = 0.20
```

```
Product.discount = 0.50
```

Situation	Recommended approach
Simple attribute change	Product.discount = 0.50 directly
Need validation or side effects	@classmethod with logic inside
Protecting instance attributes	@property with a setter

Common bug patterns



AT ALL NETWORKS

The mutable class attributes

A **mutable attribute** (list, dict, set) should never be defined as a class attribute **by accident** – but when the sharing is intentional and controlled, it becomes a powerful pattern.

Use case	Example
Registry of all instances	tracking all products, users, connections
Shared cache	storing already computed results
Configuration shared by all	a list of allowed categories for all products
Connection pool	a list of available DB connections shared across all requests

Attributes



AT ALL NETWORKS

Decision guide to choose which attribute to use.

Most of the time, you'll use **instance attributes** and only reach for class attributes when you have a conscious reason to share state across all instances.

Situation	Use
Value shared across all instances	class attribute
Value specific to each object	instance attribute
Mutable value (list, dict)	most of the time instance attribute
A counter of how many objects exist	class attribute
Default value for an attribute	instance attribute via <code>__init__</code>
A constant tied to the class	class attribute

Instance methods



AT ALL NETWORKS

```
class Animal:
    _all_animals = []

    def __init__(self, name, tricks):
        self.name = name
        self.tricks = tricks

    def display_tricks(self):
        print(self.tricks)

ani1 = Animal(name="Léo", tricks=["Roar"])
ani2 = Animal(name="Gigi", tricks=["Run fast"])

ani1.tricks.append("Play dead")
ani1.display_tricks()
```

An **instance method** is a method associated with an **object** (an instance of the class).

It always takes **self** as its **first parameter**, which gives it access to the instance's attributes and other methods.

These methods are used to perform actions on an object or manipulate its data.

class methods



AT ALL NETWORKS

```
class Animal:
    _all_animals = []

    def __init__(self, name, tricks):
        self.name = name
        self.tricks = tricks

    @classmethod
    def display_all_animals(cls):
        print(cls._all_animals)

ani1 = Animal(name="Léo", tricks=["Roar"])
ani2 = Animal(name="Gigi", tricks=["Run fast"])

ani1.tricks.append("Play dead")
ani1.display_tricks()
```

A **class method** is a method associated with the **class itself**, not a specific instance.

It is **decorated** with **@classmethod** and always takes **cls** as its first parameter, which gives it access to class attributes and other class methods.

class methods



AT ALL NETWORKS

Common use cases

Use case	When you need it
Alternative constructor	creating objects from different input formats
Read/modify class attributes	counters, shared config, registries
Singleton	guaranteeing only one instance exists
Inheritance-aware factory	creating the right subclass type automatically

class methods



AT ALL NETWORKS

The alternative constructor use case

A **class method** can be used as an **alternative constructor** to allow the creation of an object from a chosen input format.

The **pattern is always the same** – @classmethod **receives some input, transforms it, and calls cls(...)** to create the object. The input shape depends on where the data comes from.

```
@classmethod
def from_dict(cls, data: dict):
    """From a dictionary – common when parsing JSON or a DB row"""
    return cls(title=data["title"], author=data["author"], isbn=data["isbn"], pages=data["pages"])

@classmethod
def from_string(cls, text: str):
    """From a formatted string – common when parsing a CSV line"""
    title, author, isbn, pages = text.split(",")
    return cls(title=title, author=author, isbn=isbn, pages=int(pages))

@classmethod
def from_tuple(cls, data: tuple):
    """From a tuple – common when unpacking a raw DB row"""
    return cls(*data)
```

static methods



AT ALL NETWORKS

A **static method** is a **utility method** that belongs to the class but has **no access to the class or its instances**. It is decorated with **@staticmethod** and takes no self or cls parameter.

Use it when the **logic is related to the class** conceptually but doesn't need any class or instance data to do its job.

```
@staticmethod
def is_valid_isbn(isbn: str) -> bool:
    """Validates an ISBN format – needs no class or instance data."""
    digits = isbn.replace(__old: "-", __new: "")
    return len(digits) == 13 and digits.isdigit()
```

special methods



Always define `__repr__` first. It works everywhere as a fallback.
`__str__` is for the user-friendly version.



AT ALL NETWORKS

`__str__` and `__repr__`

These **special methods** control **how your object is represented as a string**.

`__str__` is for the **end user**, called when you use `print()` or `str()`. It should return something **readable and friendly**.

`__repr__` is for the **developer**, called in the Python shell, in logs, and inside collections. It should return something **unambiguous and ideally reconstructable**. (you could copy-paste the output and recreate the object)

*The `!r` inside the f-string adds quotes around string values automatically – it calls `repr()` on each value, which is **what makes the output reconstructable**.*

```
def __str__(self):
    status = "available" if self._available else "borrowed"
    return f'"{self.title}" by {self.author} [{status}]'

def __repr__(self):
    return f"Book(title={self.title!r}, author={self.author!r}, isbn={self.isbn!r})"
```

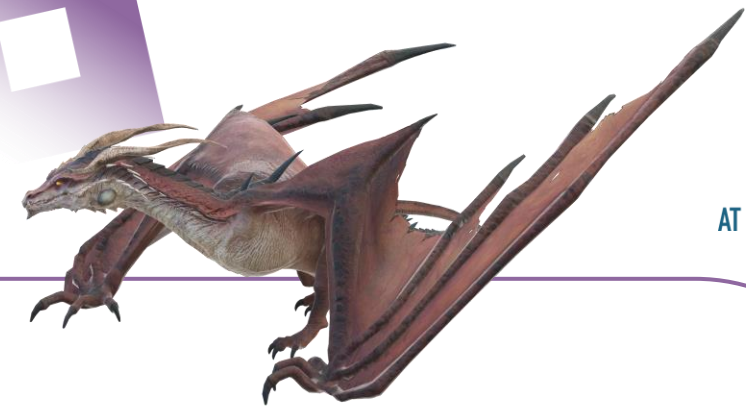
methods



AT ALL NETWORKS

Method	Decorator	First parameter	Access to	Use it when
Instance method	none	self	instance	you need to read or modify instance attributes
Class method	@classmethod	cls	class only	you need to read or modify a class attribute
Static method	@staticmethod	nothing	neither	pure utility, no access to class or instance needed

Exercise



AT ALL NETWORKS

Exercise

You are building the foundation of a mythical creature stable management system. Create a Creature class that represents a mythical creature in the stable.

The class must have:

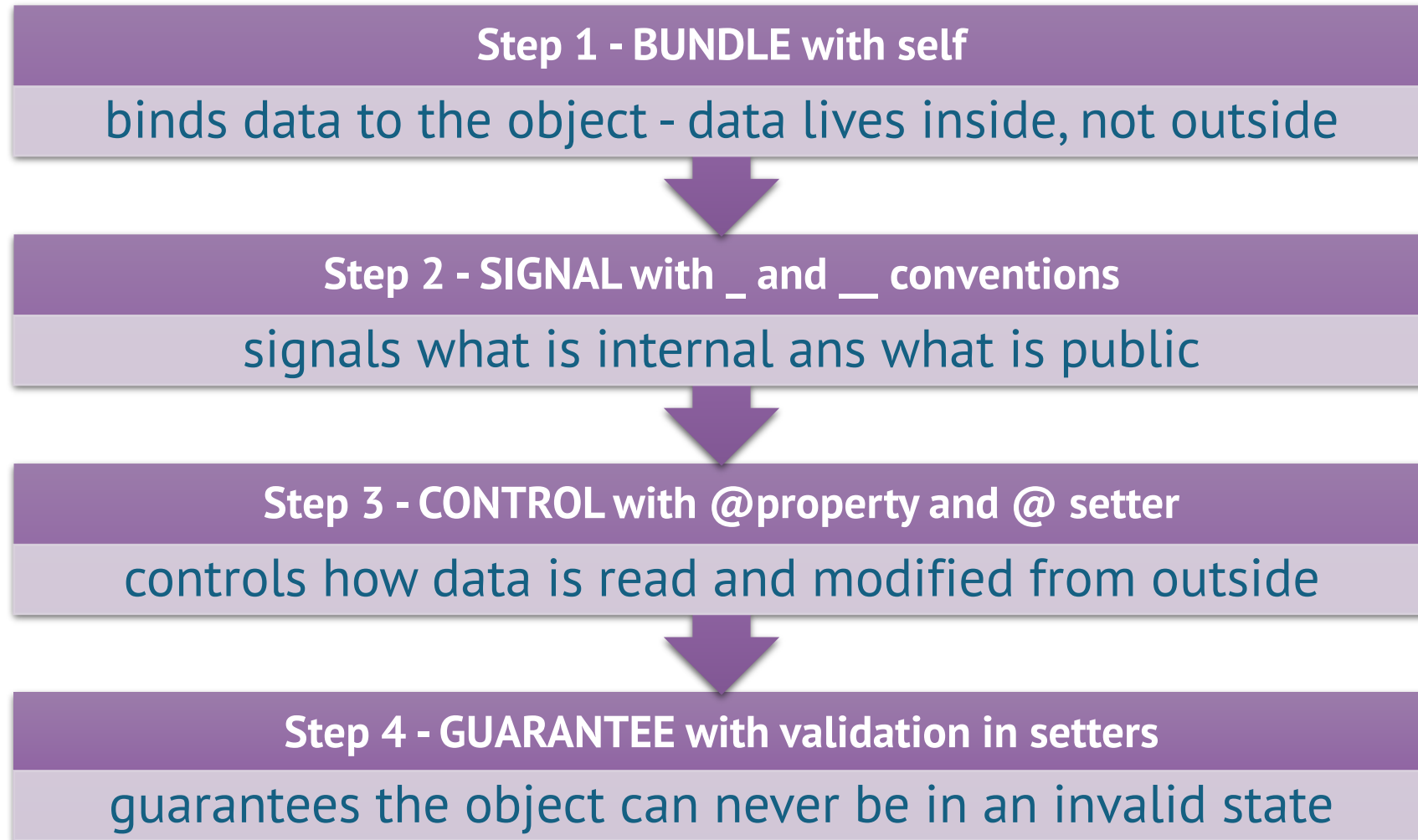
- A class attribute `_total_creatures` initialized to 0 that counts every creature registered in the stable.
- An `__init__` method that takes `name`, `species`, `origin` and `power_level` as parameters and stores them as instance attributes. `__init__` must also set a `_in_stable` attribute to `True` by default and increment `_total_creatures` by 1 each time a creature is registered.
- A `send_on_mission()` method that sets `_in_stable` to `False` – the creature is sent on a mission.
- A `return_to_stable()` method that sets `_in_stable` back to `True`
- A `__str__` method that returns a readable string like: Frostbite the Ice Dragon (origin: Nordic Realms) [in stable]

Encapsulation



AT ALL NETWORKS

Encapsulation is bundling data and behavior inside an object and controlling what the outside world can see and modify.



Encapsulation



AT ALL NETWORKS

Step 1 - Bundle

Bundling means that an **object owns** both its **data** (attributes) and the **logic** that operates on that data (methods), **they travel together**, and the object is responsible **for managing itself**.

```
class Member:
    """Represents a library member."""

    def __init__(self, member_id: int, name: str, email: str) -> None:
        self.member_id = member_id
        self.name = name
        self.__email = email          # private
        self._borrowed_books: list[Book] = []
```



***Bundling** is the **foundation** that makes the other 3 steps of **encapsulation** possible.*

Encapsulation



AT ALL NETWORKS

Step 2 - Signal

Use `_` and `__` to **communicate what is internal**. Other developers know what they should and shouldn't touch.

Name mangling

Prevents accidental name conflicts in classes, mainly during **inheritance**. It applies to attributes or methods starting with **two leading underscores** `__` and not ending with double underscores.

```
class Book:
    def __init__(self, title, author, pages):
        self.title = title          # public
        self._author = author       # protected
        self.__pages = pages        # private — name mangling applied here

b1 = Book(title="Clean Code", author="Robert C. Martin", pages=431)
```

Python **automatically renames** such identifiers internally to avoid clashes.

```
# What you write          # What Python stores internally
self.__pages              →      self._Book__pages
```

Convention	Meaning	Enforced by Python?
name	fully public	no restriction
_name	protected — internal use, subclasses ok	no, just a convention
__name	private — triggers name mangling	partially

Encapsulation



AT ALL NETWORKS



The **pythonic way** of **getters** and **setters** lets you start with plain attributes and add validation later without changing code outside the class

Step 3 - Control

Without **protection** on the attributes, anyone can set values that are not valid. Use **@property** and **@setter** to **intercept reads and writes**. You decide what gets in and what comes out.

"Getters" & "Setters", the Python way

```
def get_pages(self):  
    return self.__pages  
  
def set_pages(self, value):  
    if value <= 0:  
        raise ValueError("Pages must be positive")  
    self.__pages = value
```



@property => getter and @attr.setter => setter

```
@property  
def pages(self) -> int:  
    return self.__pages  
  
@pages.setter  
def pages(self, value: int) -> None:  
    if value <= 0:  
        raise ValueError(f"Pages must be positive, got {value}")  
    self.__pages = value
```



What you write	What Python calls	Use it when
@property	getter	you want to control how an attribute is read
@x.setter	setter	you want to validate before writing
no setter defined	/	the attribute should be read-only
computed property	getter only	the value is derived from other attributes

Encapsulation



AT ALL NETWORKS

Step 3 - Control

You can **access automatically** the getter and setter of the attribute by using the name of the attribute, in the class as well.

```
book.pages = 100
```

```
self.pages = pages
```



*! If you put the same attribute name in the `__init__` and in the setter that is called automatically, there will be an infinite recursion. You can use the `_` prefix in the setter and **not** in the `__init__`*

```
def __init__(self, title: str, author: str, isbn: str, pages: int) -> None:
    """
    __init__ method, the initializer of the class.
    Contains instance attributes and sets the initial state of the object.
    """

    # Public attributes
    self.title = title
    self.author = author
    # Protected: internal use, but accessible, _attr is a convention
    self._isbn = isbn
    # Private: name-mangled to _Book__pages
    self.pages = pages
    self.available = True
```

No prefix in the `__init__`, the setter will be called.

```
@property
def pages(self) -> int:
    """Getter - called when you read b1.pages"""
    return self.__pages

@pages.setter
def pages(self, value: int) -> None:
    """Setter - called when you write b1.pages = something"""
    if not isinstance(value, int):
        raise TypeError(f"Pages must be an integer, got {type(value).__name__}")
    if value <= 0:
        raise ValueError(f"Pages must be positive, got {value}")
    self.__pages = value
```

`_` or `__` prefix in the setter, so it doesn't call himself.

Encapsulation



AT ALL NETWORKS

Step 4 - Guarantee

Add **validation logic inside the setter**. The object is now responsible for its own **integrity** — nobody can put it in an invalid state from outside.

A setter with validation moves the crash to the **exact moment the bad value is introduced** — not later, not somewhere else.

```
@pages.setter
def pages(self, value: int) -> None:
    """Setter — called when you write b1.pages = something"""
    if not isinstance(value, int):
        raise TypeError(f"Pages must be an integer, got {type(value).__name__}")
    if value <= 0:
        raise ValueError(f"Pages must be positive, got {value}")
    self.__pages = value
```

The object **guarantees its own validity at all times**, no exceptions, no surprises.



*Validate only where bad values would cause **real problems**. Over-engineering goes against **Python's philosophy** !*

Attribute	Needs a setter with validation?
title — any string is fine	probably not
pages — must be a positive integer	yes
rating — must be between 0 and 5	yes
_available — only set internally	no — keep it private, no setter

Exercise



AT ALL NETWORKS

Exercise

Add class methods and static methods to the Creature class :

The stable is expanding. The stable masters now receive creature data from two new sources; a dictionary pulled from a database and a formatted string coming from an ancient scroll. They also need a way to validate a creature's species before registering it.

Update your existing Creature class with the following:

- Class methods
- Add a `from_dict()` alternative constructor that creates a Creature from a dictionary with the keys `name`, `species`, `origin` and `power_level`
- Add a `from_string()` alternative constructor that creates a Creature from a comma-separated string like "Frostbite,Ice Dragon,Nordic Realms,95"
- Add a `get_total_creatures()` class method that returns the current value of `_total_creatures`
- Static method
- Add an `is_valid_species()` static method that takes a species name and returns `True` if it is in the following list of recognized species, `False` otherwise: ["Dragon", "Ice Dragon", "Phoenix", "Griffin", "Unicorn", "Basilisk"]



AT ALL NETWORKS

We know the way!